

EE8 CPU

Lab: Instruction Decoder & Control Unit

1 Objectives

By the end of this lab you will have built a **combinational logic circuit** that takes a 4-bit opcode from the instruction word and produces the control signals (bit flags) that will drive every other component in your CPU.

You will:

1. Understand why a CPU needs control signals and what each one does.
2. Extract the opcode from the 16-bit instruction word.
3. Design and implement a combinational decoder that sets bit flags based on the opcode.
4. Test your decoder against every opcode in the EE8 instruction set.

2 Background

2.1 Where are we?

So far you have built:

- **PIPO registers** — 8-bit storage from D flip-flops.
- **ROM** — 16 rows of 16-bit instruction words.
- **Program Counter** — T flip-flop counter that selects the current ROM address, with reset (all bits to 0) and an unconditional jump override.

Right now your ROM outputs a 16-bit instruction word every clock cycle, but nothing *interprets* it. The instruction word is just a bundle of 16 wires carrying ones and zeros. The CPU needs logic that looks at those bits and decides:

- Should a register be written to?
- Should the ALU add, subtract, or do something else?
- Is this a jump instruction?
- Should the CPU halt?

This is the job of the **instruction decoder** and **control unit**.

2.2 What is a control signal?

A **control signal** (or **bit flag**) is a single wire that is either 0 or 1. It acts as an on/off switch for a specific part of the CPU.

For example, you already have one control signal: the **Jump** flag on your program counter. When **Jump** = 1, the PC loads the jump target address instead of incrementing. When **Jump** = 0, the PC increments normally ($PC = PC + 1$).

The control unit's job is to produce *all* of these flags simultaneously, based purely on the opcode. It is **combinational logic** — no flip-flops, no clock, no memory. The opcode goes in, the flags come out.

Design Note

In a real CPU (ARM, RISC-V, x86) the control unit works the same way — it decodes the instruction and asserts control signals that steer the datapath. The only difference is scale: those CPUs have hundreds of signals; the EE8 has a handful. You are building the same fundamental circuit.

3 Part A: Extracting the Opcode

The 16-bit instruction word from ROM has a specific structure. Regardless of instruction type (R-type, I-type, or J-type), the **opcode is always bits [15:12]** — the top 4 bits.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode				(varies by instruction type)											

Task A

Use a **splitter** in Logisim to extract bits [15:12] from the ROM output. This gives you a 4-bit opcode signal.

Your circuit should have:

- **Input:** instruction[15:0] (16-bit wire from ROM)
- **Output:** opcode[3:0] (4-bit wire)

You will extract the other fields (Rs1, Rs2, Rd, immediate, address) in a later lab when you build the register file and datapath. For now, the opcode is all the control unit needs.

4 Part B: The Control Signals

Your control unit will produce the following bit flags. Each flag is a single output wire (1 bit).

4.1 Core Signals (Required)

Signal	Width	Purpose
RegWrite	1 bit	Register write enable. When 1, the result of the current instruction will be written into a register. When 0, no register is modified. <i>Example:</i> ADD writes a result to Rd, so RegWrite = 1. JMP does not write to any register, so RegWrite = 0.
ImmSel	1 bit	Immediate select. When 1, the data written to a register comes from the immediate field in the instruction (bits [7:0]) rather than from the ALU. Only the LDI instruction uses this.
Jump	1 bit	Unconditional jump. When 1, the PC is loaded with the jump target address instead of PC+1. You already have this signal connected to your PC — now you need to generate it from the opcode.
Branch	1 bit	Conditional branch enable. When 1, the PC <i>may</i> be overridden with the branch target — but only if the branch condition is met (checked against a register value). Used by JZ and JNZ .
Halt	1 bit	Halt. When 1, the PC stops incrementing and the CPU freezes. Only the HALT instruction sets this.

Table 1: Core control signals — you must implement all of these.

Important

The **Jump** signal you are generating here replaces the manual jump flag you may have used for testing previously. After this lab, the jump flag will be *automatically* set whenever the opcode is 1011 (**JMP**).

4.2 Extension Signals (Optional)

Once the core signals are working, you may add these for a more complete control unit. These will become useful in later labs.

Signal	Width	Purpose
BranchCond	1 bit	Selects which branch condition to test. 0 = JZ (jump if register is zero), 1 = JNZ (jump if register is not zero). Only meaningful when Branch = 1.
ALUOp[3:0]	4 bits	Tells the ALU which operation to perform (add, subtract, AND, etc.). For most R-type instructions this can be a direct passthrough of the opcode. You will connect this when you build the ALU.
FlagWrite	1 bit	When 1, the comparison result is written to R3. Only LT and EQ use this. Needed so that normal ALU instructions (ADD , etc.) do not accidentally overwrite R3.
SysErr	1 bit	Illegal instruction. Set to 1 when the opcode is 1110 (reserved). Latched externally to halt the CPU and raise the alarm.

Table 2: Extension control signals — implement these for a more complete design.

5 Part C: The Truth Table

The control unit is a **combinational logic function**: the 4-bit opcode is the input, and the control signals are the outputs. Before you build gates, you must first work out the truth table.

Task C1 — Fill in the Truth Table

Complete the table below. For each opcode, decide whether each control signal should be 0 or 1.

Ask yourself for each instruction:

- Does this instruction write a result to a register? → **RegWrite**
- Does this instruction use an immediate value instead of the ALU? → **ImmSel**
- Does this instruction jump unconditionally? → **Jump**
- Does this instruction branch conditionally? → **Branch**
- Does this instruction stop the CPU? → **Halt**

Opcode	Binary	Mnemonic	<i>RegWrite</i>	<i>ImmSel</i>	<i>Jump</i>	<i>Branch</i>	<i>Halt</i>
0	0000	ADD					
1	0001	SUB					
2	0010	AND					
3	0011	OR					
4	0100	XOR					
5	0101	SHL					
6	0110	SHR					
7	0111	MOV					
8	1000	LT					
9	1001	EQ					
10	1010	LDI					
11	1011	JMP					
12	1100	JZ					
13	1101	JNZ					
14	1110	(reserved)					
15	1111	HALT					

Table 3: Control signal truth table — fill this in before building your circuit.

Hint

Look at the patterns in your completed table. You should notice that several signals are only set for one or two opcodes, while others are set for a whole range. This tells you how complex the gate logic needs to be for each signal.

For example, `Halt` is 1 for only *one* opcode. That means you need a circuit that outputs 1 *only* when the opcode is exactly 1111. How many gates does that take?

6 Part D: Build the Control Unit

Now turn your truth table into a Logisim circuit using combinational logic gates.

6.1 Approach

For each output signal, you have a column of 0s and 1s in your truth table. You need to design a logic circuit that produces that exact column given the 4-bit opcode input.

There are two standard approaches:

1. **Sum of products (SOP):** For each row where the output is 1, write a minterm (an AND of the 4 opcode bits or their complements). OR all the minterms together. This is what you would get from a Karnaugh map.
2. **Direct pattern matching:** Some signals are 1 for a single opcode value. For these, you just need to detect that specific 4-bit pattern using AND and NOT gates. For example, `Halt` is 1 only when opcode = 1111, so:

$$\text{Halt} = \text{opcode}[3] \text{ AND } \text{opcode}[2] \text{ AND } \text{opcode}[1] \text{ AND } \text{opcode}[0]$$

That is a single 4-input AND gate.

Design Note

You may simplify your logic using Karnaugh maps, Boolean algebra, or by spotting patterns in the truth table. Any correct combinational implementation is acceptable.

Some questions to consider while simplifying:

- Are there signals where you only need to check one or two opcode bits?
- Are there groups of opcodes that share the same output pattern?
- Would it be simpler to compute a signal as NOT of something else?

Task D — Build the Circuit

Create a new Logisim subcircuit called `control_unit`.

Interface:

- **Input:** `opcode[3:0]` (4 bits)
- **Outputs:** `RegWrite`, `ImmSel`, `Jump`, `Branch`, `Halt` (1 bit each)

Build the combinational logic for each output using AND, OR, NOT (and NAND, NOR, XOR if you wish). **Do not use a ROM component** — the point of this exercise is to design the logic yourself.

If you are also implementing the extension signals, add them as additional outputs: `BranchCond`, `ALUOp[3:0]`, `FlagWrite`, `SysErr`.

7 Part E: Testing Your Decoder

Task E — Verify All 16 Opcodes

Test your control unit by manually setting the 4-bit opcode input to each value from 0000 to 1111 and checking that the output flags match your truth table.

Use **probes** or **LEDs** on each output to see the flag values.

Record your results in the table below (or take a screenshot for each opcode).

Opcode	Mnemonic	<i>RegWrite</i>	<i>ImmSel</i>	<i>Jump</i>	<i>Branch</i>	<i>Halt</i>	Pass?
0000	ADD						☐
0001	SUB						☐
0010	AND						☐
0011	OR						☐
0100	XOR						☐
0101	SHL						☐
0110	SHR						☐
0111	MOV						☐
1000	LT						☐
1001	EQ						☐
1010	LDI						☐
1011	JMP						☐
1100	JZ						☐
1101	JNZ						☐
1110	(reserved)						☐
1111	HALT						☐

Table 4: Test results — fill in the observed outputs and tick if they match your truth table.

Hint

If a signal is wrong for a specific opcode, trace back through your gates. Check:

- Are your NOT gates on the correct opcode bits?
- Did you accidentally swap two opcode bit positions in a splitter?
- Is the gate type correct (AND vs OR)?

8 Part F: Connecting the Jump Flag to Your PC

You already have a working program counter with an unconditional jump input. Now you can connect the **Jump** output from your control unit to that input, so the CPU automatically jumps when it encounters a **JMP** instruction.

Task F — Integration Test

1. Connect the ROM output through your splitter to extract `opcode[3:0]`.
2. Feed the opcode into your control unit.
3. Connect the Jump output to your PC's jump-enable input.
4. Connect the Halt output to your PC's halt/freeze input (if you have one), or use it to gate the clock.
5. Load a simple test program into your ROM:

```
; Test program: JMP should loop forever between addresses 0 and 2
; You should be able to load this into your ROMs.
; Don't worry about LDI doing nothing for now.
0: LDI R0 42      ; 1010 00 00 00101010 -> 0xA02A
1: LDI R1 10      ; 1010 00 01 00001010 -> 0xA10A
2: JMP 0          ; 1011 00 000000 0000 -> 0xB000
3: HALT          ; 1111 000000000000 -> 0xF000
```

Expected behaviour:

- The PC should count 0, 1, 2, 0, 1, 2, ... (looping).
- Address 3 (**HALT**) should never be reached.
- The Jump LED should light up only when the PC is at address 2.
- The RegWrite LED should light up at addresses 0 and 1 (the **LDI** instructions).

9 Deliverables

1. **Completed truth table** (Table 3).
2. **Logisim subcircuit:** `control_unit` with correct inputs/outputs.
3. **Test results** showing all 16 opcodes produce the correct flags.
4. **Integration test** demonstrating the Jump and Halt signals working with your existing PC and ROM.

Extension deliverables (optional):

- BranchCond signal implemented and tested.
- ALUOp[3:0] output (can be a passthrough of the opcode for R-type instructions, don't-care for non-ALU instructions).
- FlagWrite signal for LT/EQ.
- SysErr signal for opcode 1110.

10 Summary

You have built the “brain” of the CPU — the part that interprets instructions and tells every other component what to do. Every signal in your control unit is a simple combinational function of the 4-bit opcode.

In the next lab, you will build the **register file**: four registers with multiplexers for reading (Rs1, Rs2) and a decoder for write-enable (Rd). The **RegWrite** signal you built today will drive that write-enable logic.

Design Note

The instruction decoder pattern you used here — extract a field, match patterns, produce control signals — is the same pattern used in every processor ever made, from the 1971 Intel 4004 to modern ARM and RISC-V cores. The logic gets bigger, but the idea is identical.